

Отчёт по лабораторной работе №5

Расширенная настройка HTTP-сервера Apache

Цыкунова Екатерина Михайловна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Конфигурирование HTTP-сервера для работы через протокол HTTPS	6
2.2	Конфигурирование HTTP-сервера для работы с PHP	15
2.3	Внесение изменений в настройки внутреннего окружения виртуальной машины	18
3	Вывод	21
4	Контрольные вопросы	22

Список иллюстраций

2.1	Генерация закрытого ключа RSA и самоподписанного сертификата	8
2.2	Настройка виртуального хоста <code>www.emsikunova.net</code> для работы через HTTPS	11
2.3	Разрешение службы HTTPS в <code>firewalld</code> и перезапуск Apache	12
2.4	Предупреждение Firefox о самоподписанном сертификате	13
2.5	Проверка работы веб-сервера <code>www.emsikunova.net</code> через HTTPS	14
2.6	Просмотр сведений о самоподписанном сертификате	14
2.7	Создание файла <code>index.php</code> с вызовом функции <code>phpinfo</code>	16
2.8	Настройка владельца файлов, восстановление контекстов SELinux и перезапуск Apache	17
2.9	Проверка работы PHP на веб-сервере	17
2.10	Копирование конфигурации веб-сервера, закрытого ключа и сертификата в каталог <code>provisioning</code>	19
2.11	Скорректированный сценарий автоматической настройки HTTP-сервера	20

Список таблиц

1 Цель работы

Приобретение практических навыков по расширенному конфигурированию HTTP сервера Apache в части безопасности и возможности использования PHP.

2 Выполнение лабораторной работы

2.1 Конфигурирование HTTP-сервера для работы через протокол HTTPS

На виртуальной машине `server` открываю терминал и перехожу в режим суперпользователя командой `sudo su`. Для хранения закрытого криптографического ключа создаю каталог `/etc/pki/tls/private` командой `mkdir -p /etc/pki/tls/private/`. Параметр `-p` позволяет создать необходимую структуру каталогов и не выдаёт ошибку, если каталог уже существует.

Для обеспечения доступа к закрытым ключам через стандартный путь `/etc/ssl/private` создаю символическую ссылку командой `ln -s /etc/pki/tls/private/ /etc/ssl/private`. После этого перехожу в каталог `/etc/pki/tls/private/`, в котором выполняю генерацию закрытого ключа и самоподписанного сертификата (см. рис. Рисунок 2.1).

Для генерации используется команда:

```
openssl req -x509 -nodes -newkey rsa:2048 -keyout www.emcikunova.net.key -out www.emcikunova.net.crt
```

Команда `openssl req` запускает создание запроса и сертификата. Параметр `-x509` указывает OpenSSL сразу сформировать самоподписанный сертификат стандарта X.509 вместо обычного запроса на подпись сертификата. Параметр `-nodes` отключает защиту закрытого ключа парольной фразой, благодаря чему веб-сервер Apache сможет автоматически прочитать ключ при запус-

ке без необходимости ручного ввода пароля. Параметр `-newkey rsa:2048` создаёт новый закрытый ключ RSA длиной 2048 бит. С помощью `-keyout www.emcikunova.net.key` задаётся имя файла закрытого ключа, а параметр `-out www.emcikunova.net.crt` определяет имя создаваемого сертификата.

Во время создания сертификата OpenSSL запрашивает сведения о владельце. В поле `Country Name` указываю код страны RU, в `State or Province Name` — Russia, в `Locality Name` — Moscow. В качестве организации и подразделения указываю логин emcikunova. В поле `Common Name` задаю доменное имя emcikunova.net, а в поле электронной почты — emcikunova@emcikunova.net.

Большое количество символов +, . и *, отображаемых OpenSSL перед запросом параметров сертификата, соответствует процессу генерации криптографического материала. Возврат приглашения командной строки без сообщения об ошибке свидетельствует об успешном создании файлов закрытого ключа и сертификата.

Для дальнейшего использования сертификата Apache файл сертификата размещается в каталоге сертификатов `/etc/pki/tls/certs`, в то время как закрытый ключ остаётся в защищённом каталоге `/etc/pki/tls/private`.

[illegible]

Рисунок 2.1: Генерация закрытого ключа RSA и самоподписанного сертификата

После создания криптографического ключа и сертификата изменяю конфигурацию виртуального веб-сервера `www.emcikunova.net`. Для этого редактирую файл `/etc/httpd/conf.d/www.emcikunova.net.conf`. В нём настраиваю перенаправление запросов HTTP на HTTPS и отдельный виртуальный хост, обслуживающий защищённые соединения на TCP-порту 443 (см. рис. Рисунок 2.2).

Конфигурационный файл имеет следующий вид:

```
<VirtualHost *:80>

    ServerAdmin webmaster@emcikunova.net

    DocumentRoot /var/www/html/www.emcikunova.net

    ServerName www.emcikunova.net

    ServerAlias www.emcikunova.net

    ErrorLog logs/www.emcikunova.net-error_log

    CustomLog logs/www.emcikunova.net-access_log common

    RewriteEngine on
```

```

    RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]
</VirtualHost>

<IfModule mod_ssl.c>
<VirtualHost *:443>
    SSLEngine on
    ServerAdmin webmaster@emcikunova.net
    DocumentRoot /var/www/html/www.emcikunova.net
    ServerName www.emcikunova.net
    ServerAlias www.emcikunova.net
    ErrorLog logs/www.emcikunova.net-error_log
    CustomLog logs/www.emcikunova.net-access_log common
    SSLCertificateFile /etc/ssl/certs/www.emcikunova.net.crt
    SSLCertificateKeyFile /etc/ssl/private/www.emcikunova.net.key
</VirtualHost>
</IfModule>

```

Строка `<VirtualHost *:80>` открывает описание виртуального хоста, принимающего обычные HTTP-соединения на порту 80 на всех сетевых интерфейсах сервера.

Директива `ServerAdmin webmaster@emcikunova.net` устанавливает адрес электронной почты администратора веб-сервера, который может использоваться Apache при формировании служебных страниц и сообщений об ошибках.

Директива `DocumentRoot /var/www/html/www.emcikunova.net` задаёт корневой каталог веб-сайта. Именно в этом каталоге находятся файлы, выдаваемые клиенту при обращении к виртуальному хосту.

Строка `ServerName www.emcikunova.net` определяет основное доменное имя виртуального веб-сервера. Apache использует значение заголовка `Host` HTTP-запроса для выбора подходящего виртуального хоста.

Директива `ServerAlias www.emcikunova.net` определяет дополнительное имя, по которому может быть доступен виртуальный хост. В данной конфигурации оно совпадает с основным именем сервера.

Строка `ErrorLog logs/www.emcikunova.net-error_log` задаёт файл журнала ошибок для виртуального хоста. В него Apache записывает ошибки обработки запросов и другие диагностические сообщения.

Директива `CustomLog logs/www.emcikunova.net-access_log common` задаёт отдельный журнал запросов к сайту. Параметр `common` определяет стандартный формат записи сведений о клиенте, запросе, коде ответа и объёме переданных данных.

Команда `RewriteEngine on` активирует механизм преобразования URL модуля `mod_rewrite` для данного виртуального хоста.

Строка `RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]` выполняет перенаправление любого запроса, пришедшего по HTTP, на аналогичный адрес с использованием HTTPS. Выражение `^(.*)$` соответствует всему пути запроса, `%{HTTP_HOST}` сохраняет имя запрошенного хоста, а `$1` добавляет исходный путь. Флаг `R=301` задаёт постоянное HTTP-перенаправление с кодом состояния 301 `Moved Permanently`, а флаг `L` сообщает модулю `mod_rewrite`, что после применения данного правила дальнейшая обработка правил не требуется.

Строка `</VirtualHost>` завершает описание виртуального хоста, работающего на порту 80.

Конструкция `<IfModule mod_ssl.c>` задаёт условный блок конфигурации. Вложенные директивы обрабатываются только при наличии загруженного модуля Apache `mod_ssl`, обеспечивающего поддержку TLS/SSL.

Строка `<VirtualHost *:443>` создаёт виртуальный хост для защищённых HTTPS-соединений на TCP-порту 443.

Директива `SSLEngine on` включает механизм SSL/TLS для данного виртуального хоста.

Директивы `ServerAdmin`, `DocumentRoot`, `ServerName`, `ServerAlias`, `ErrorLog` и

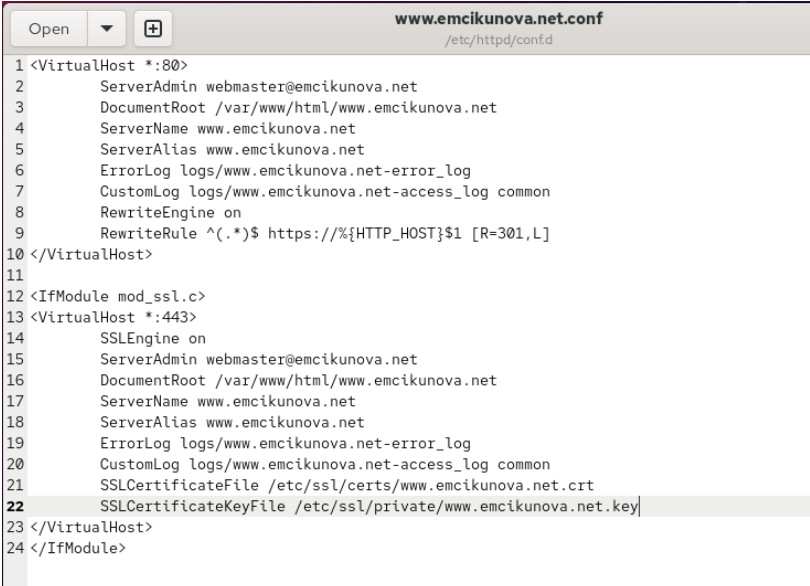
CustomLog выполняют те же функции, что и в HTTP-виртуальном хосте: задают администратора, каталог сайта, доменное имя и отдельные журналы доступа и ошибок.

Строка `SSLCertificateFile /etc/ssl/certs/www.emcikunova.net.crt` указывает Apache путь к файлу сертификата, который сервер передаёт клиенту в процессе установления TLS-соединения.

Строка `SSLCertificateKeyFile /etc/ssl/private/www.emcikunova.net.key` задаёт путь к соответствующему закрытому ключу. Закрытый ключ используется сервером при выполнении криптографических операций во время установки защищённого соединения и не должен передаваться клиентам.

Строка `</VirtualHost>` завершает описание HTTPS-виртуального хоста, а `</IfModule>` завершает условный блок конфигурации модуля `mod_ssl`.

В результате обращения к `http://www.emcikunova.net` Apache принимает запрос на порту 80 и перенаправляет браузер на `https://www.emcikunova.net`, после чего соединение обслуживается виртуальным хостом на порту 443.



```
1 <VirtualHost *:80>
2     ServerAdmin webmaster@emcikunova.net
3     DocumentRoot /var/www/html/www.emcikunova.net
4     ServerName www.emcikunova.net
5     ServerAlias www.emcikunova.net
6     ErrorLog logs/www.emcikunova.net-error_log
7     CustomLog logs/www.emcikunova.net-access_log common
8     RewriteEngine on
9     RewriteRule ^(.*)$ https://%{HTTP_HOST}%1 [R=301,L]
10 </VirtualHost>
11
12 <IfModule mod_ssl.c>
13 <VirtualHost *:443>
14     SSLEngine on
15     ServerAdmin webmaster@emcikunova.net
16     DocumentRoot /var/www/html/www.emcikunova.net
17     ServerName www.emcikunova.net
18     ServerAlias www.emcikunova.net
19     ErrorLog logs/www.emcikunova.net-error_log
20     CustomLog logs/www.emcikunova.net-access_log common
21     SSLCertificateFile /etc/ssl/certs/www.emcikunova.net.crt
22     SSLCertificateKeyFile /etc/ssl/private/www.emcikunova.net.key
23 </VirtualHost>
24 </IfModule>
```

Рисунок 2.2: Настройка виртуального хоста `www.emcikunova.net` для работы через HTTPS

После настройки Apache разрешаю входящие HTTPS-соединения в межсетевом экране `firewalld`. Командой `firewall-cmd --add-service=https` добавляю службу `https` в текущую конфигурацию межсетевого экрана. Полученное сообщение `success` подтверждает успешное применение правила.

Чтобы разрешение сохранилось после перезагрузки виртуальной машины, выполняю `firewall-cmd --add-service=https --permanent`. Эта команда добавляет службу HTTPS в постоянную конфигурацию `firewalld`. Затем командой `firewall-cmd --reload` перечитываю постоянную конфигурацию и применяю внесённые изменения.

После настройки межсетевого экрана перезапускаю Apache командой `systemctl restart httpd`, чтобы сервер перечитал изменённый файл виртуального хоста и начал обслуживать HTTPS-соединения (см. рис. Рисунок 2.3). Отсутствие сообщений об ошибках после команды `systemctl restart httpd` указывает, что служба была перезапущена и конфигурация принята.

```
[root@server.emcikunova.net conf.d]#  
[root@server.emcikunova.net conf.d]# firewall-cmd --add-service=https  
success  
[root@server.emcikunova.net conf.d]# firewall-cmd --add-service=https --permanent  
success  
[root@server.emcikunova.net conf.d]# firewall-cmd --reload  
success  
[root@server.emcikunova.net conf.d]# systemctl restart httpd  
[root@server.emcikunova.net conf.d]#
```

Рисунок 2.3: Разрешение службы HTTPS в `firewalld` и перезапуск Apache

Для проверки настройки перехожу на виртуальную машину `client` и открываю в браузере веб-сервер `www.emcikunova.net`. Браузер автоматически переходит к защищённому соединению, после чего Firefox выводит страницу `Warning: Potential Security Risk Ahead` (см. рис. Рисунок 2.4).

На странице указано, что сертификат сайта не является доверенным, поскольку он самоподписанный. Браузер выводит код ошибки `MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT`. Появление данного предупреждения является ожидаемым результатом: сертификат был создан непосредственно на сервере с помощью `OpenSSL` и не подписан центром сертификации,

которому доверяет браузер.

Нажимаю кнопку **Advanced . . .**, после чего становится доступна возможность просмотра сертификата и продолжения работы с сайтом через **Accept the Risk and Continue**. Для лабораторного окружения разрешаю использование данного сертификата.

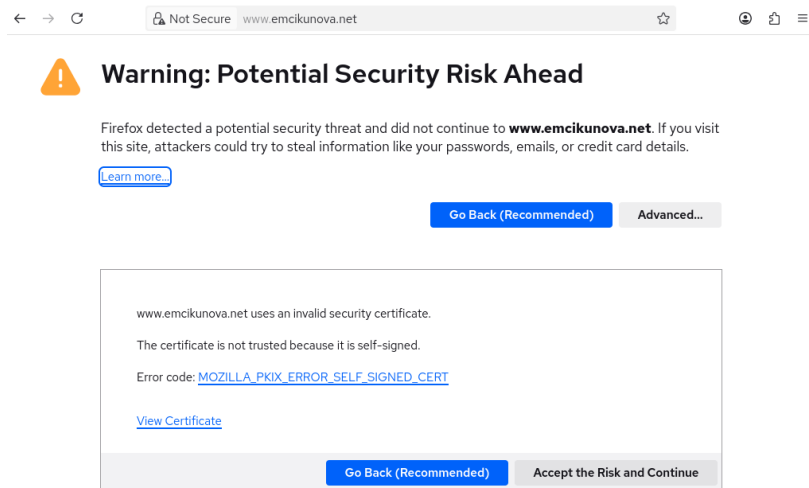


Рисунок 2.4: Предупреждение Firefox о самоподписанном сертификате

После добавления исключения браузер открывает страницу виртуального хоста `www.emcikunova.net` по протоколу HTTPS (см. рис. Рисунок 2.5). В адресной строке отображается имя `www.emcikunova.net`, а в содержимом страницы выводится сообщение `Welcome to the www.emcikunova.net server.`

Успешное отображение страницы подтверждает, что Apache принимает HTTPS-соединения на порту 443, правило `firewallld` разрешает соответствующий сетевой трафик, а виртуальный хост использует созданный сертификат и закрытый ключ. Кроме того, автоматический переход к защищённой версии сайта подтверждает работоспособность правила перенаправления, заданного директивой `RewriteRule`.

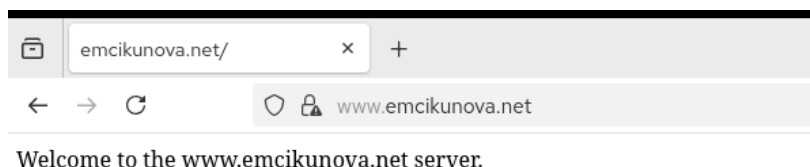


Рисунок 2.5: Проверка работы веб-сервера www.emcikunova.net через HTTPS

Далее просматриваю сведения о сертификате в Firefox (см. рис. Рисунок 2.6). В разделе Subject Name отображаются данные, введённые во время работы команды `openssl req`: страна RU, регион Russia, город Moscow, организация emcikunova, подразделение emcikunova, общее имя emcikunova.net и адрес электронной почты emcikunova@emcikunova.net.

В разделе Issuer Name отображаются аналогичные сведения об издателе сертификата. Совпадение данных владельца и издателя связано с использованием самоподписанного сертификата: роль центра сертификации в данном случае выполняет сам владелец созданного сертификата.

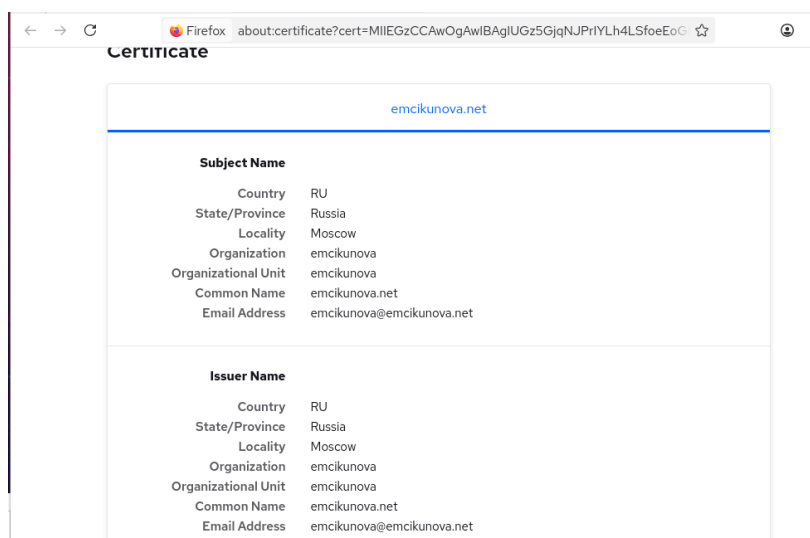


Рисунок 2.6: Просмотр сведений о самоподписанном сертификате

2.2 Конфигурирование HTTP-сервера для работы с PHP

Для добавления поддержки PHP на виртуальной машине server устанавливаю пакет php командой:

```
dnf -y install php
```

Параметр -y автоматически подтверждает установку необходимых пакетов и зависимостей.

После установки PHP перехожу в каталог веб-сайта `/var/www/html/www.emcikunova.net/`. Имеющийся файл `index.html` переименовываю в `index.php` командой `mv index.html index.php`, после чего открываю его для редактирования и заменяю содержимое PHP-скриптом (см. рис. Рисунок 2.7):

```
<?php
    phpinfo();
?>
```

Открывающий тег `<?php` сообщает интерпретатору о начале PHP-кода. Функция `phpinfo()` генерирует диагностическую HTML-страницу, содержащую сведения об установленной версии PHP, конфигурации интерпретатора, серверном API, подключённых расширениях и параметрах среды. Закрывающий тег `?>` завершает блок PHP-кода.

Использование функции `phpinfo()` удобно для проверки корректности интеграции PHP с веб-сервером, поскольку успешное отображение созданной ею страницы подтверждает, что файл `.php` обрабатывается интерпретатором, а не передаётся клиенту как обычный текстовый файл.

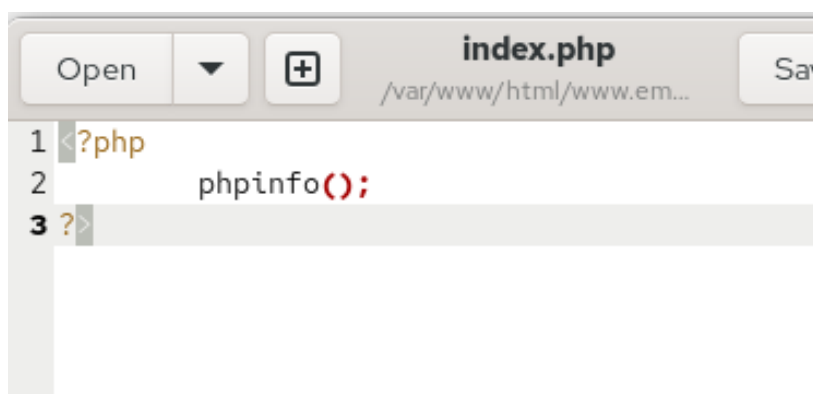


Рисунок 2.7: Создание файла index.php с вызовом функции phpinfo

После создания PHP-страницы корректирую владельца файлов веб-сервера командой `chown -R apache:apache /var/www` (см. рис. Рисунок 2.8). Параметр `-R` обеспечивает рекурсивное изменение владельца для каталога `/var/www` и всего его содержимого. Пользователь и группа `apache` соответствуют учётной записи, от имени которой работает HTTP-сервер.

Затем восстанавливаю корректные контексты безопасности SELinux командами `restorecon -vR /etc` и `restorecon -vR /var/www/`. Параметр `-R` выполняет рекурсивную обработку каталогов, а `-v` включает отображение изменяемых объектов.

При выполнении `restorecon -vR /etc` отображается сообщение о переназначении контекста файла `/etc/NetworkManager/system-connections/eth1.nmconnection`, что подтверждает фактическую проверку и восстановление SELinux-контекстов. Команда для `/var/www/` возвращает приглашение командной строки без дополнительного вывода, следовательно, для находящихся там объектов дополнительных изменений отображать не потребовалось.

После изменения прав доступа и восстановления контекстов SELinux выполняю `systemctl restart httpd`. Перезапуск необходим для применения установленной поддержки PHP и проверки работы веб-сервера с обновлённым содержимым сайта.

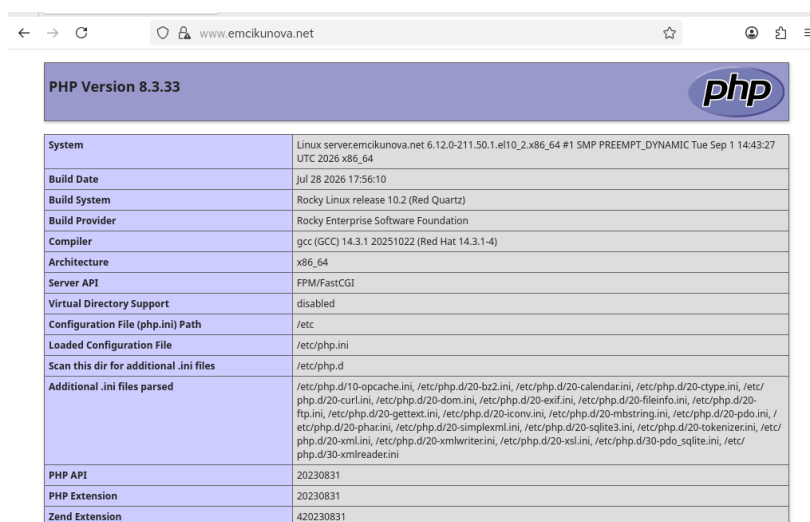
```
[root@server.emcikunova.net conf.d]# cd /var/www/html/www.emcikunova.net/
[root@server.emcikunova.net www.emcikunova.net]# mv index.html index.php
[root@server.emcikunova.net www.emcikunova.net]# gedit index.php
[root@server.emcikunova.net www.emcikunova.net]# chown -R apache:apache /var/www
[root@server.emcikunova.net www.emcikunova.net]# restorecon -vR /etc
Relabeled /etc/NetworkManager/system-connections/eth1.nmconnection from unconfined_u:object_r:user_tmp_t:s0 to unconfined_u:object_r:NetworkManager_etc_rw_t:s0
[root@server.emcikunova.net www.emcikunova.net]# restorecon -vR /var/www/
[root@server.emcikunova.net www.emcikunova.net]# systemctl restart httpd
[root@server.emcikunova.net www.emcikunova.net]#
```

Рисунок 2.8: Настройка владельца файлов, восстановление контекстов SELinux и перезапуск Apache

На виртуальной машине client повторно открываю адрес `www.emcikunova.net`. Вместо прежнего статического приветствия браузер отображает страницу, сформированную функцией `phpinfo()` (см. рис. Рисунок 2.9).

В верхней части страницы указана версия PHP 8.3.33. В таблице также отображается информация о системе Linux `server.emcikunova.net`, архитектуре `x86_64`, используемом серверном API `FPM/FastCGI`, основном конфигурационном файле `/etc/php.ini` и каталоге дополнительных конфигурационных файлов `/etc/php.d`.

Появление страницы `phpinfo()` подтверждает, что PHP установлен на сервере и PHP-скрипты корректно обрабатываются при обращении к сайту через Apache. Одновременно страница загружена с адреса `www.emcikunova.net` через настроенный ранее HTTPS-виртуальный хост.



PHP Version 8.3.33	
System	Linux server.emcikunova.net 6.12.0-211.50.1.el10_2.x86_64 #1 SMP PREEMPT_DYNAMIC Tue Sep 1 14:43:27 UTC 2026 x86_64
Build Date	Jul 28 2026 17:56:10
Build System	Rocky Linux release 10.2 (Red Quartz)
Build Provider	Rocky Enterprise Software Foundation
Compiler	gcc (GCC) 14.3.1 20251022 (Red Hat 14.3.1-4)
Architecture	x86_64
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc
Loaded Configuration File	/etc/php.ini
Scan this dir for additional .ini files	/etc/php.d
Additional .ini files parsed	/etc/php.d/10-opcache.ini, /etc/php.d/20-bz2.ini, /etc/php.d/20-calendar.ini, /etc/php.d/20-ctype.ini, /etc/php.d/20-curl.ini, /etc/php.d/20-dom.ini, /etc/php.d/20-exif.ini, /etc/php.d/20-fileinfo.ini, /etc/php.d/20-ftp.ini, /etc/php.d/20-gettext.ini, /etc/php.d/20-iconv.ini, /etc/php.d/20-mbstring.ini, /etc/php.d/20-pdo.ini, /etc/php.d/20-phar.ini, /etc/php.d/20-simplexml.ini, /etc/php.d/20-sqlite3.ini, /etc/php.d/20-tokenizer.ini, /etc/php.d/20-xml.ini, /etc/php.d/20-xmlwriter.ini, /etc/php.d/20-xsl.ini, /etc/php.d/30-pdo_sqlite.ini, /etc/php.d/30-xmlreader.ini
PHP API	20230831
PHP Extension	20230831
Zend Extension	420230831

Рисунок 2.9: Проверка работы PHP на веб-сервере

2.3 Внесение изменений в настройки внутреннего окружения виртуальной машины

После завершения ручной настройки сохраняю созданные конфигурационные файлы и содержимое веб-сервера в каталоге provisioning проекта Vagrant. Это позволяет использовать подготовленную конфигурацию при последующем автоматическом развёртывании виртуальной машины.

Конфигурационные файлы Apache копирую командой:

```
cp -R /etc/httpd/conf.d/* /vagrant/provision/server/http/etc/httpd/conf.d
```

На экране появляются запросы на перезапись существующих файлов, в том числе autoindex.conf, fcgid.conf, manual.conf, ssl.conf, userdir.conf, welcome.conf, а также конфигурационных файлов виртуальных хостов. Перезапись подтверждаю вводом у, чтобы в каталоге проекта находились актуальные версии конфигурации.

Содержимое каталога веб-сервера сохраняю командой:

```
cp -R /var/www/html/* /vagrant/provision/server/http/var/www/html/
```

При копировании также подтверждаю замену ранее существующих файлов.

Для хранения криптографических данных в структуре provisioning создаю каталоги:

```
mkdir -p /vagrant/provision/server/http/etc/pki/tls/private  
mkdir -p /vagrant/provision/server/http/etc/pki/tls/certs
```

После этого копирую закрытый ключ:

```
cp -R /etc/pki/tls/private/www.emcikunova.net.key /vagrant/provision/server/http/etc/pki/t
```

и сертификат:

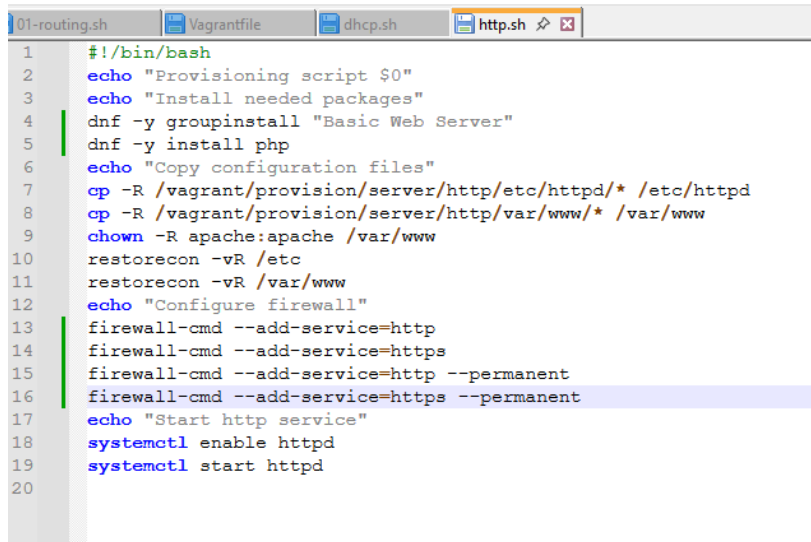
```
cp -R /etc/pki/tls/certs/www.emcikunova.net.crt /vagrant/provision/server/http/etc/pki/t
```

В результате в каталоге проекта сохраняются конфигурация Apache, содержимое веб-сайта, закрытый ключ и сертификат, необходимые для воспроизведения настроенного окружения виртуальной машины (см. рис. Рисунок 2.10).

```
[root@server.emcikunova.net www.emcikunova.net]# cp -R /etc/httpd/conf.d/* /vagrant/provision/server/http/etc/httpd/conf.d/
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/autoindex.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/fcgid.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/manual.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/README'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/server.emcikunova.net.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/ssl.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/userdir.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/welcome.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/www.emcikunova.net.conf'? y
[root@server.emcikunova.net www.emcikunova.net]# cp -R /var/www/html/* /vagrant/provision/server/http/var/www/html/
cp: overwrite '/vagrant/provision/server/http/var/www/html/server.emcikunova.net/index.html'? y
[root@server.emcikunova.net www.emcikunova.net]# mkdir -p /vagrant/provision/server/http/etc/pki/tls/private
[root@server.emcikunova.net www.emcikunova.net]# mkdir -p /vagrant/provision/server/http/etc/pki/tls/certs
[root@server.emcikunova.net www.emcikunova.net]# cp -R /etc/pki/tls/private/www.emcikunova.net.key /vagrant/provision/server/http/etc/pki/tls/private/
[root@server.emcikunova.net www.emcikunova.net]# cp -R /etc/pki/tls/certs/www.emcikunova.net.crt /vagrant/provision/server/http/etc/pki/tls/certs/
[root@server.emcikunova.net www.emcikunova.net]#
```

Рисунок 2.10: Копирование конфигурации веб-сервера, закрытого ключа и сертификата в каталог provisioning

На завершающем этапе корректирую сценарий автоматической настройки /vagrant/provision/server/http.sh (см. рис. Рисунок 2.11). Скрипт содержит последовательность команд, необходимых для подготовки HTTP-сервера при создании виртуальной машины.

A screenshot of a terminal window with a tabbed interface. The active tab is titled 'http.sh'. The terminal displays a series of 20 lines of shell script. The script starts with a shebang, followed by echo statements for logging, then dnf commands to install 'Basic Web Server' and 'php'. It then copies configuration files from a vagrant provision directory to the system's httpd and www directories, sets permissions, restores permissions, and finally configures the firewall to allow http and https traffic permanently. The script ends with enabling and starting the httpd service.

```
1  #!/bin/bash
2  echo "Provisioning script $0"
3  echo "Install needed packages"
4  dnf -y groupinstall "Basic Web Server"
5  dnf -y install php
6  echo "Copy configuration files"
7  cp -R /vagrant/provision/server/http/etc/httpd/* /etc/httpd
8  cp -R /vagrant/provision/server/http/var/www/* /var/www
9  chown -R apache:apache /var/www
10 restorecon -vR /etc
11 restorecon -vR /var/www
12 echo "Configure firewall"
13 firewall-cmd --add-service=http
14 firewall-cmd --add-service=https
15 firewall-cmd --add-service=http --permanent
16 firewall-cmd --add-service=https --permanent
17 echo "Start http service"
18 systemctl enable httpd
19 systemctl start httpd
20
```

Рисунок 2.11: Скорректированный сценарий автоматической настройки HTTP-сервера

Готовый фрагмент для продолжения отчёта:

3 Вывод

В результате работы настроена поддержка HTTPS для веб-сервера Apache: создан самоподписанный сертификат, разрешён HTTPS-трафик и настроено перенаправление с HTTP на HTTPS.

Также установлена поддержка PHP и проверена работа PHP-страницы через браузер. Конфигурация сервера сохранена в каталоге `provision`, а скрипт `http.sh` дополнен установкой PHP и настройкой HTTPS.

4 Контрольные вопросы

1. В чём отличие HTTP от HTTPS?

HTTP (HyperText Transfer Protocol) — протокол передачи данных между клиентом и веб-сервером. При использовании обычного HTTP передаваемые данные сами по себе не шифруются, поэтому при наличии доступа к сетевому трафику их потенциально можно перехватить или изменить. По умолчанию HTTP использует TCP-порт 80.

HTTPS (HyperText Transfer Protocol Secure) представляет собой HTTP, работающий поверх защищённого соединения TLS. Передаваемые между браузером и веб-сервером данные шифруются, а сертификат сервера используется для подтверждения его подлинности. По умолчанию HTTPS использует TCP-порт 443.

2. Каким образом обеспечивается безопасность контента веб-сервера при работе через HTTPS?

Безопасность при работе через HTTPS обеспечивается протоколом TLS. При установлении соединения сервер передаёт клиенту цифровой сертификат, содержащий сведения о сервере и его открытый ключ. Клиент проверяет сертификат и в процессе TLS-соединения стороны согласовывают параметры шифрования и формируют сеансовые ключи.

После установления защищённого соединения HTTP-запросы и ответы передаются в зашифрованном виде. Это обеспечивает конфиденциальность передаваемых данных и затрудняет их чтение посторонними лицами.

TLS также обеспечивает контроль целостности данных, позволяя обнару-

жить изменение информации во время передачи. Проверка цифрового сертификата позволяет клиенту удостовериться, что он устанавливает соединение с требуемым веб-сервером. В данной лабораторной работе использовался самоподписанный сертификат, поэтому браузер не мог автоматически подтвердить доверие к его издателю и выводил соответствующее предупреждение.

3. Что такое сертификационный центр? Приведите пример.

Сертификационный центр, или удостоверяющий центр (Certificate Authority, CA), — организация, которая выпускает и подписывает цифровые сертификаты. Перед выдачей сертификата сертификационный центр проверяет право заявителя использовать указанное доменное имя или выполняет более расширенную проверку в зависимости от типа сертификата.

Браузеры и операционные системы содержат хранилище доверенных корневых сертификатов сертификационных центров. Если сертификат веб-сервера подписан доверенным центром и успешно проходит проверку, браузер может подтвердить подлинность сертификата без дополнительного вмешательства пользователя.

Примером сертификационного центра является Let's Encrypt. Он бесплатно выпускает TLS-сертификаты для доменных имён и широко применяется для организации защищённых HTTPS-соединений.